

4-2 · Apache Iceberg 레이크하우스 구축 · 정리·회고 0.5H

정리·회고

명령 열 개를 한 장으로, 4-3 출발 상태 확인

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

30분 뒤에는

- ① Iceberg SQL 열 개를 「하려는 일」 기준으로 정리한다
- ② 4-3 을 위한 상태(컨테이너 4개 · orders 4행 · raw CSV)를 확인한다
- ③ 자주 겪는 오류 네 가지의 원인과 해결을 안다
- ④ 4-3 MinIO-Iceberg 파이프라인에서 할 일을 미리 본다

오늘 만든 `demo.lakehouse.orders` 는 4주차 내내 씁니다 — 4-3 적재, 4-4 Trino 조회

Iceberg SQL — 만들고 조회하기

하려는 일	SQL
테이블 만들기	CREATE TABLE demo.db.t (...) USING iceberg
파티션과 함께	CREATE TABLE ... PARTITIONED BY (days(order_date))
스냅샷 이력	SELECT * FROM demo.db.t.snapshots
시점 조회 (id)	SELECT * FROM demo.db.t VERSION AS OF 스냅샷id
시점 조회 (시각)	SELECT * FROM demo.db.t TIMESTAMP AS OF '...'

demo.db.t 는 오늘의 demo.lakehouse.orders — 카탈로그.데이터베이스.테이블 세 단계

Iceberg SQL — 되돌리기 · 바꾸기 · 정리하기

하려는 일	SQL
롤백	CALL demo.system.rollback_to_snapshot('db.t', id)
컬럼 추가	ALTER TABLE demo.db.t ADD COLUMN c STRING
파일 목록	SELECT * FROM demo.db.t.files
스냅샷 정리	CALL demo.system.expire_snapshots('db.t', TIMESTAMP '...')
작은 파일 병합	CALL demo.system.rewrite_data_files('db.t')

expire_snapshots 로 정리한 시점 이전으로는 못 돌아간다 — 보관 기간(보통 7~30일)을 정하는 것이 운영 결정

4-3 을 위한 상태 확인

네 줄로 확인 — 컨테이너 · 행 수 · 원천 CSV

폴더	cd ~/lakehouse
컨테이너	docker compose ps
행 수	spark-sql -e "SELECT count(*) FROM demo.lakehouse.orders;"
원천 CSV	docker compose exec mc mc ls --recursive local/raw

확인 포인트: 컨테이너 4개 Up · orders 가 4행 · raw/orders/ 에 orders_2026-08-01.csv 가 있으면 4-3 준비 끝

여덟 개를 스스로 확인해 보세요

- 1 파일만 쌓았을 때 안 되는 것을 세 가지 이상 말할 수 있다
- 2 카탈로그 → 스냅샷 → 매니페스트 → 데이터 파일 사슬을 그릴 수 있다
- 3 컨테이너 4개(minio·mc·rest·spark-iceberg)가 모두 Up 이다
- 4 REST 카탈로그(8181) 응답과 Spark 카탈로그 설정을 직접 확인했다
- 5 MinIO 에서 metadata/ · data/ 파일이 실제로 생긴 것을 확인했다
- 6 snapshots 에서 id 를 찾아 VERSION AS OF 조회에 성공했다
- 7 rollback_to_snapshot 으로 되돌리고 이력에 남는 것을 확인했다
- 8 ADD COLUMN 뒤 parquet 개수가 안 늘어나는 것을 확인했다

증상으로 찾는 원인과 해결

증상	원인	해결
CREATE TABLE 에서 NoSuchBucket	warehouse 버킷을 만들지 않았다	mc mb -p local/warehouse 뒤 다시 시도
rest:8181 connection refused	REST 카탈로그 컨테이너가 안 떴거나 죽었다	compose ps · logs rest → compose up -d
VERSION AS OF 에서 Cannot find snapshot	없는 id 이거나 따옴표로 감쌌다	snapshots 의 id 를 따옴표 없는 숫자로

spark-sql 이 뜨다 죽거나 느리면 메모리 부족 — Docker Desktop → Settings → Resources → Memory 8GB 이상, 안 쓰는 컨테이너는 stop

자주 나오는 질문 네 가지

1 Iceberg 와 Delta Lake 중 뭘 배워야 하나요?

개념이 거의 같다 — Iceberg 는 엔진 중립적이라 Spark·Trino·Flink 가 함께 쓰기 좋고, Delta 는 Databricks 생태계에서 강하다

2 스냅샷이 계속 쌓이면 저장 비용이 늘지 않나요?

늘어난다 — `expire_snapshots` 로 보관 기간을 정해 정리한다, 보통 7~30일

3 UPDATE 마다 파일을 통째로 다시 쓰면 느리지 않나요?

느리다 — 대량 배치 갱신에 맞고 건건이 수정하는 업무는 여전히 RDB, 4-3 의 MERGE INTO 가 배치 갱신

4 카탈로그가 죽으면 데이터도 잃나요?

데이터 파일은 남는다 — 다만 「무엇이 현재 상태인지」를 잃으므로 카탈로그도 백업 대상

다음 회차 예고

4-3 · MinIO-Iceberg 데이터 파이프라인 구축

01

이론 1H

데이터 파이프라인
설계 · 맥등성

02

실습 1H

원천 데이터
raw 버킷 적재

03

실습 1.5H

Spark ETL 과
MERGE 적재

04

정리 0.5H

스냅샷 이력으로
적재 결과 검증

준비: 오늘 구성 그대로 docker compose up -d 로 4개 컨테이너 · spark-sql 에서 orders 를 SELECT 해 살아 있는지 확인

4-2 · 정리·회고 0.5H 마무리

다음 회차는 4-3 MinIO-Iceberg 파이프라인

raw 버킷의 CSV 를 PySpark 로 정제해 MERGE INTO 로 오늘의 orders 테이블에 중복 없이 적재합니다

✓ 랜딩 존(raw) → 정제 → 적재 단계 설계

✓ 멍등성이 파이프라인의 기본 원칙인 이유

✓ s3a 로 MinIO 의 원천 CSV 읽기

✓ MERGE INTO 업서트 · 스냅샷으로 검증

준비할 것

4개 컨테이너 Up · orders SELECT 확인